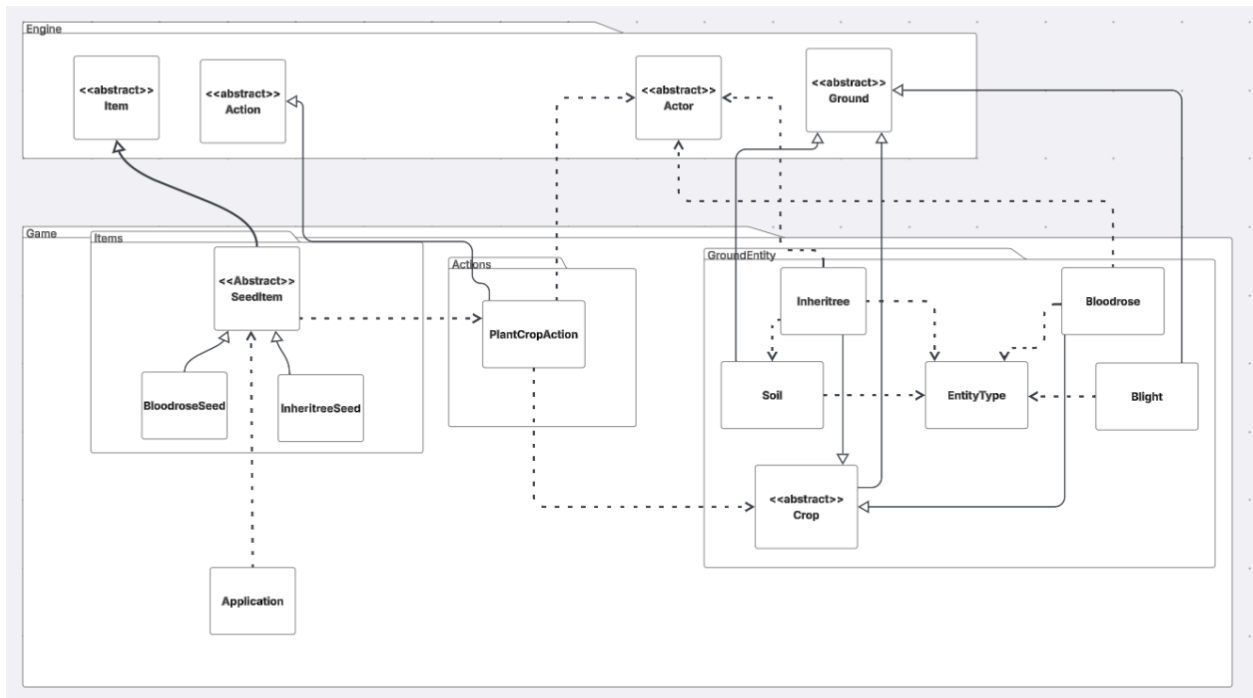
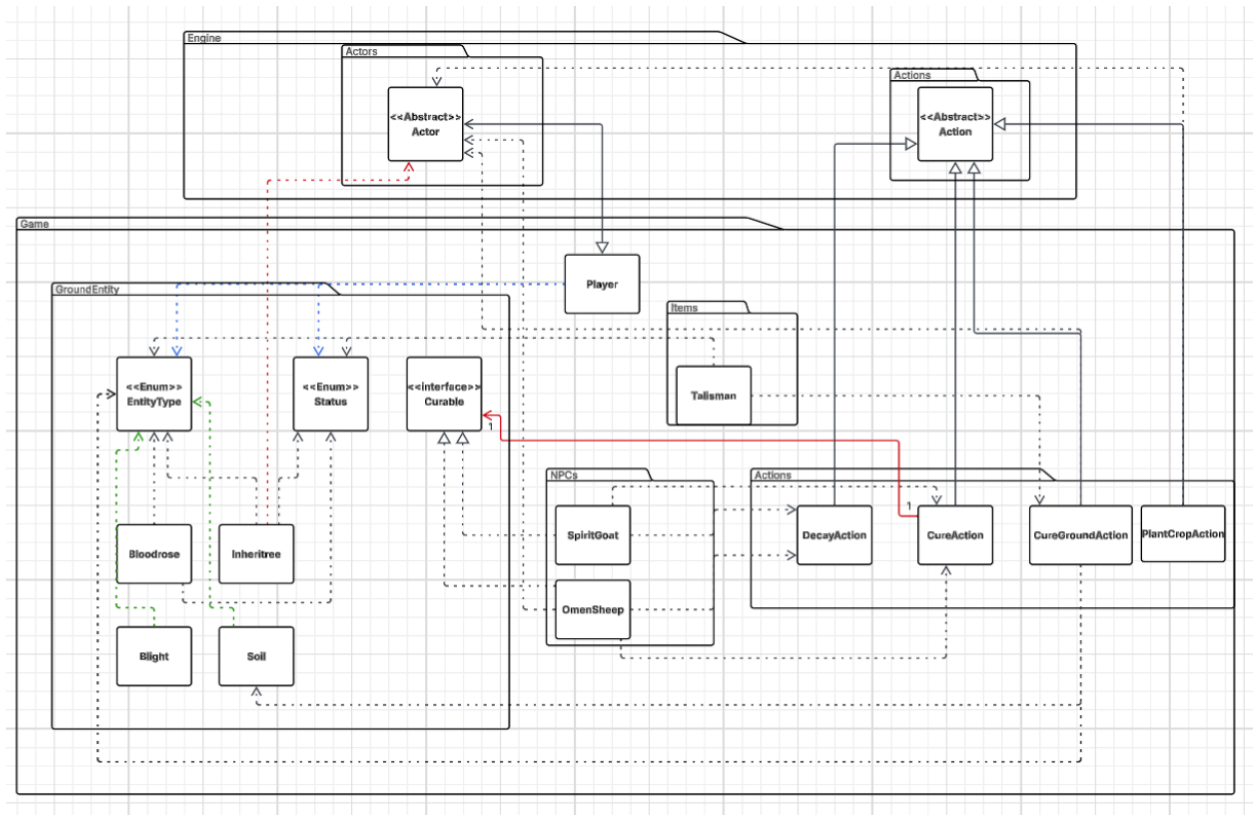


REQ 2 UML Class Diagram:



REQ 3 UML Class Diagram:



Design Rationale

REQ 1

Design 1: Create SpiritGoat and OmenSheep as independent classes extending Actor

Pros	Cons
Simple to implement for a small number of NPCs.	Repeated code for health management, behavior setup, and allowable actions.
Maximum flexibility per NPC — each can be totally custom.	Lacks reuse — violates DRY (Don't Repeat Yourself)
Doesn't require an extra abstraction layer.	Any shared change must be copied to all NPCs manually

Design 2 (Chosen): Create an abstract NPC class to encapsulate common features, and have SpiritGoat and OmenSheep extend it.

Pros	Cons
Promotes code reuse: shared logic (HP, allowable actions) lives in one place.	Tight coupling to NPC class — changes here affect all child classes.
Scalable: Adding new NPCs is easier (Open For extension closed for Modification)	Requires thoughtful abstraction to avoid unnecessary inheritance.
Easier maintenance: Fixes or feature additions only need to be made once (DRY)	
Supports Liskov Substitution Principle — new NPCs can be used in place of NPC.	

Explanation:

For Requirement 1, I chose design 2, creating non-hostile creatures SpiritGoat and OmenSheep by extending a shared abstract class NPC. This design choice promotes code reuse by centralizing shared logic for allowable actions, hitpoint tracking, and interaction with the Player. It adheres to SOLID principles, particularly the Liskov Substitution Principle, as both subclasses can be used wherever an NPC is expected. It also supports easy extensibility for future NPC types. Each NPC defines its own lifespan and behavior logic, but common interaction mechanisms—such as being attackable or hostile—are managed in the abstract class or via interfaces. This modular and abstracted approach reduces duplication and simplifies maintenance across different types of non-hostile entities.

REQ 2

A.

Implementation of BloodRoseSeed and InheritreeSeed

Design 1: Create BloodRoseSeed and InheritreeSeed classes that extend the Item class and implement a Plantable interface to implement planting logic.

Pros	Cons
Lightweight — any Item could be made plantable without needing to subclass.	Logic for planting becomes more scattered.
Flexible — a single item could theoretically grow different things.	Need to check dynamically at runtime: "Is this item Plantable?" (instanceof) Harder to control planting behavior

Design 2: (Chosen) Create a new Abstract class; SeedItem, which extends the item and is then inherited by BloodRoseSeed and InheritreeSeed classes.

Pros	Cons
Clear and explicit — a seed is a special kind of Item. Easy to extend (Open/Closed) Keeps the responsibility of "what you grow" close to the seed. (SRP)	Requires a new subclass for every new seed.

Explanation:

For this requirement I chose to create an Abstract class that extends Items, SeedItem. This is implemented by the concrete classes BloodRoseSeed and inheritreeSeed. This aligns with Single Responsibility as Items unrelated to planting remain separated, it also supports Open/Closed as properties of SeedItems can easily be inherited without affecting old ones. This also supports Liskov Substitution because Anywhere an Item is expected, a SeedItem can be used which helps utilise the Engine logic for Items, like portable, icon etc.

B. How should the planting action happen?

Design 1: (Chosen) Custom PlantCropAction class

Pros	Cons
Planting is an action given to the player and therefore utilises the given Engine as it is a subclass of Action.	Requires writing a new Action.
Keeps the responsibility of "what you grow" close to the seed. (Single Responsibility)	Adds complexity to allowableActions() for items.

Design 2: Treat planting like dropping an item with extra effects (e.g., Drop seed → If dropped on Soil, it automatically grows.)

Pros	Cons
Reuses DropItemAction, no new Action needed.	No explicit "Plant" option — might confuse players
Very simple to implement.	Does Not work well for future seeds that might have different planting behaviors.

Explanation:

Planting should be an explicit player action, not an implicit drop mechanic, therefore I chose design 2. This is done via creating a PlantCropAction, which is added to the player's action via SeedItem. This reduces code duplication but assumes every SeedItem can be planted. This supports SRP as now only PlantCropAction handles planting logic, as well as OCP, since new SeedItem types can easily be planted.

C. How should the Crop's post-planting behavior work (e.g., Inheritree healing, Bloodrose harming)?

Design 1: (chosen) Create a new Crop abstract class that extends Ground with methods bloom() and Tick() then create concrete classes of different Plants, extending the Crop class.

Pros	Cons
Easy to customize one-off effects when a crop first blooms	Limited inheritance to only 1 abstract class per concrete class, limits the development of functionality of each individual concrete class.
Encourages separation of concerns: each Crop knows what it does. (SRP)	

Design 2: Use a Plant interface, create concrete classes of different Plants by extending Ground abstract class in the engine and using the interface to implement post plant behaviour

Pros	Cons
Plant interface utilises	More repeated code, methods enforced that have the same logic would still need to be implemented in all concrete classes that use the interface

Explanation:

For this requirement I created Crop-an abstract class extending Ground-which is inherited via concrete BloodRose and Inheritree class's. This Crop class is very similar to an Interface, having empty methods that are overridden via each concrete subclass to implement its own bloom and tick logic. I avoided using an interface as it was easier for SeedItem to transform into a Crop when planted since it didn't need an Association to an Interface, only using a dependency. This design also supports Single Responsibility as each Crop is responsible for its own behavior without bloating the base Ground class and supports Open/Closed since New crops can extend Crop without modifying it. To aid in implementing Bloom and Tick logic I also utilised the inbuilt Capabilities of Ground through a new Enum class EntityType which distinguished blight (Cursed).

REQ 3

A. Implementation of Stamina loss to farmer when performing specific Actions

Design 1: Add stamina cost logic in Each plant class and use instanceof to check if Farmer is selecting Action. Then decrease stamina of Farmer

Pros	Cons
Direct and easy to implement for a small number of plant types. Specific stamina costs can be hard-coded per plant.	Violates Single Responsibility Principle (SRP): plant classes now manage stamina too. instanceOf checks break Open/Closed Principle (OCP); adding new types means modifying existing code.

Design 2: (chosen) Decrease stamina in PlantActionClass using a check stamina cost method from crop abstract class.

Pros	Cons
------	------

Stamina is managed at the Action level, where it belongs (SRP)
crops can define their stamina cost without modifying plantaction
(Open/Closed) Centralized stamina logic, easier to maintain and debug.

Requires a new method in the Crop class and careful documentation.
Slightly more setup needed when creating new crops.

Explanation:

To implement this requirement I utilised the Engines capabilities of the players Stamina and added the Stamina decrementing logic in my PlantCropAction class to support SRP, as only the specific Action is responsible for Stamina decrementing. This is supported via a dependency to Crop which supports OCP, LSP and DIP. I can easily add new crops stamina amount (OCP) without changing PlantCropAction logic, additionally any new Crop can be substituted without breaking PlantCropAction since it relies on the crop Abstract class on specific Crop subclasses (LSP). Furthermore I support DIP since my action depends on abstraction (crop stamina behaviour) rather than concrete implementations within subclasses and I can avoid Instanceof/type.

B. Animals OmenSheep and SpiritGoat decay after a set amount of turns

Design: (chosen) Create a new DecayAction class extending Action, with a counter attribute in NPC subclasses that is decremented each turn through the engine's playTurn method. When it reaches 0, the DecayAction is triggered, added to allowableActions and triggered. This also supports OCP, since I rely on a single dependency to Crop

Pros

DecayAction handles removal logic.
(SRP)
Easily extensible to new types of NPCs that need to decay or don't decay
(Open/Closed)

Cons

Increases complexity of the Actor's playTurn method by requiring counter management.
Need to ensure DecayAction is prioritized correctly among other actions. (ordered)

Explanation:

For this requirement I created a new Decay Action which extends actions to handle removing my NPC's when their decay timer is met. The NPC subclasses handle their own lifespan logic through a decrement being called at the start of each PlayTurn, supporting SRP in both cases. This solution is also Open/Closed since any new NPC can be removed through a decayAction being added to their allowable actions. I could have implemented this logic within the NPC base class to reduce duplicated code, however I decided to keep it separate since I assume future NPCs will be added which are not Cursed and therefore do not decay. This solution also supports DIP since I utilise the Engines Playturn method and actions which rely on abstractions themselves; checking for general behaviour, not specific to NPC types.

C. Talisman can be used to cure Animals and ground.

Design 1: Utilise allowableActions from Items abstract class, and have two methods for ground and Actor types using InstanceOf. Talisman then returns a CureAction if the owner is on ground or near an animal.

Pros	Cons
Fast and simple to implement Minimal setup for a small number of curable types.	violates Open/Closed Principle — must modify Talisman if new curable types are added. Harder to maintain and scale as more curable entities are added. Breaks Interface Segregation Principle (ISP): Talisman has to know too much about different types

Design 2: NPC subclasses and Blight implements a Curable interface. Talisman detects via capability/status.

Pros	Cons
respects Open/Closed Principle — new curables only need to implement Curable, no changes to Talisman. Extensible and clean: future creatures or grounds can easily become curable.	Requires more upfront work: Curable interface, capability system setup, status management. Needs to use Downcasting to pass Curable into CureAction as its not an attribute of Talisman

Design 3: (Chosen) NPC implements Curable interface with Cure() method; CureAction is split into CureGroundAction and CureAction.

Pros	Cons
Clear responsibilities: each Curable object knows how to cure itself. new curable types just implement Curable. (Open/Closed) any Curable entity can be passed to CureAction without type-checking.	Requires careful management of CureAction variants, possibly OverEngineering.

Explanation:

For this requirement I created a new Curable interface that is implemented by OmenSheep and SpiritGoat classes. The talisman item interacts with the NPC's through a Status capability added

to the player. This gives the following logic; Engine asks NPC what the player can do to you, it returns CureAction if they possess the Talisman, then Engine tells player it can cure NPC. This supports SRP, as animals are responsible for telling the player they can be cured, but the CureAction logic is separated via a Curable interface. When the player uses the Talisman on a SpiritGoat or OmenSheep, a CureAction is created and executed which handles Cure logic through the Curable interface's Cure() method. This also supports Interface Segregation Principle, as Curable isolates curing behaviour without forcing NPCs to implement unrelated methods. This also supports new NPC's that may or may not be Curable (Open and Closed Principle). It also supports Liskov Substitution Principle since Curable is passed to CureAction, anywhere a Curable is expected, a SpiritGoat or OmenSheep can be used safely. Furthermore To implement Blight cure logic I used the engines Capabilities statuses to add a Cursed EntityType enum to its capabilities. From this Talisman can check for Cursed entities, supporting DIP as the Talisman only depends on a Capability check and not on concrete classes like Blight or SpiritGoat. After this check it adds a CureGroundAction which can be triggered by the player. This supports SRP as CureGroundAction only handles curing ground, and Talisman is only responsible for curing the ground. This also supports OCP as new types of ground types can be added without changing Talisman or Ground, through a new EntityType capability.

D. Implement Stamina cost to cure soil

Design: (Chosen) Decrease Actor stamina in CureGroundAction

Pros	Cons
Action manages stamina cost for its specific task. (SRP) Easy to maintain and adjust stamina cost centrally in CureGroundAction. any Curable entity can be passed to CureAction without type-checking.	Slight duplication if many different "ground cure" actions later have different stamina costs (can be solved by refactoring stamina cost into Ground properties if needed).

Explanation:

To implement this requirement I simply added a stamina check and decrement inside my CureGroundAction. This supports SRP as only my action is responsible for decreasing stamina. It also supports DIP since any actor can have their stamina decreased if they call this action. (Though we only have one player capable of calling this action for now).